

Panduan Membuat Dashboard Streamlit untuk Proyek E-Commerce

Panduan ini dirancang untuk pemula total. Ikuti langkah demi langkah.

Daftar Isi

1. [Apa itu Streamlit?](#)
 2. [Instalasi](#)
 3. [Struktur File Dashboard](#)
 4. [Membuat Dashboard Langkah demi Langkah](#)
 5. [Menjalankan Dashboard di Local](#)
 6. [Deploy ke Streamlit Cloud \(Gratis\)](#)
 7. [Kode Dashboard Lengkap](#)
-

1. Apa itu Streamlit?

Streamlit adalah library Python yang memungkinkan kamu membuat web dashboard **hanya dengan Python** — tidak perlu HTML, CSS, atau JavaScript sama sekali.

Cara kerjanya sederhana:

- Kamu tulis script Python biasa
 - Streamlit otomatis mengubahnya menjadi halaman web interaktif
 - Setiap kali kamu mengubah kode, tampilannya langsung diperbarui
-

2. Instalasi

Prasyarat

Pastikan Python sudah terinstal. Cek dengan:

```
bash  
  
python --version
```

Install Streamlit dan library pendukung

Buka terminal/command prompt, lalu jalankan:

```
bash
```

```
pip install streamlit pandas matplotlib seaborn plotly
```

Cek apakah instalasi berhasil:

```
bash
```

```
streamlit --version
```

3. Struktur File Dashboard

Susun folder proyekmu seperti ini (sesuai rubrik Dicoding):

```
submission/
├── dashboard/
│   ├── main_data.csv      ← data bersih hasil analisis
│   └── dashboard.py       ← file utama Streamlit
├── data/
│   ├── orders_dataset.csv
│   └── ... (dataset asli)
├── notebook.ipynb
├── README.md
├── requirements.txt
└── url.txt                ← isi dengan link Streamlit Cloud (jika sudah deploy)
```

Membuat main_data.csv

Di akhir notebook analisismu, simpan data yang sudah dibersihkan:

```
python
```

```
# Di notebook, setelah cleaning selesai:
main_data = orders_filtered.merge(order_payments_df, on='order_id')
main_data.to_csv('dashboard/main_data.csv', index=False)
print("main_data.csv berhasil disimpan!")
```

Membuat requirements.txt

Buat file `requirements.txt` dengan isi:

```
streamlit
pandas
matplotlib
seaborn
plotly
```

4. Membuat Dashboard Langkah demi Langkah

Konsep Dasar Streamlit

Fungsi	Kegunaan	Contoh
<code>st.title()</code>	Judul besar	<code>st.title("Dashboard E-Commerce")</code>
<code>st.header()</code>	Subjudul	<code>st.header("Analisis Pembayaran")</code>
<code>st.write()</code>	Teks/data apa saja	<code>st.write("Hello World")</code>
<code>st.dataframe()</code>	Tampilkan tabel	<code>st.dataframe(df)</code>
<code>st.pyplot()</code>	Tampilkan grafik matplotlib	<code>st.pyplot(fig)</code>
<code>st.sidebar</code>	Panel filter di kiri	<code>st.sidebar.selectbox(...)</code>
<code>st.columns()</code>	Layout kolom	<code>col1, col2 = st.columns(2)</code>
<code>st.metric()</code>	Kartu angka	<code>st.metric("Total Order", "100K")</code>
<code>st.date_input()</code>	Filter tanggal	<code>st.date_input("Pilih tanggal")</code>

Komponen Filter (Sidebar)

Filter membuat dashboard menjadi interaktif. Letakkan di sidebar agar rapi:

```
python
```

```
# Di sidebar
st.sidebar.header("Filter Data")

# Filter rentang tanggal
date_range = st.sidebar.date_input(
    "Rentang Tanggal",
    value=[df['order_purchase_timestamp'].min(),
          df['order_purchase_timestamp'].max()]
)

# Filter metode pembayaran
payment_options = ['Semua'] + df['payment_type'].unique().tolist()
selected_payment = st.sidebar.selectbox("Metode Pembayaran", payment_options)
```

Menampilkan Visualisasi

Setiap grafik matplotlib/seaborn harus dibungkus dalam `plt.figure()` dan ditampilkan dengan `st.pyplot()`:

```
python

fig, ax = plt.subplots(figsize=(10, 5))
ax.bar(data['payment_type'], data['avg_payment'], color='#1976D2')
ax.set_title("Rata-rata Nilai Transaksi per Metode")
ax.set_xlabel("Metode Pembayaran")
ax.set_ylabel("Nilai (BRL)")
st.pyplot(fig)
plt.close() # Penting! Selalu close figure setelah ditampilkan
```

5. Menjalankan Dashboard di Local

1. Buka terminal/command prompt
2. Masuk ke folder submission:

```
bash

cd path/ke/folder/submission
```

3. Jalankan Streamlit:

```
bash

streamlit run dashboard/dashboard.py
```

4. Browser akan otomatis terbuka di `http://localhost:8501`

Untuk menghentikan, tekan `Ctrl + C` di terminal.

6. Deploy ke Streamlit Cloud (Gratis)

Deploy memungkinkan dashboardmu diakses siapa saja lewat internet.

Langkah-langkah:

1. Upload ke GitHub

- Buat akun di github.com jika belum ada
- Buat repository baru (public)
- Upload seluruh folder submission ke repository tersebut

2. Daftar Streamlit Cloud

- Buka share.streamlit.io
- Login dengan akun GitHub

3. Deploy

- Klik "New app"
- Pilih repository GitHub-mu
- Set "Main file path" ke: `dashboard/dashboard.py`
- Klik "Deploy!"

4. Simpan URL

- Setelah deploy selesai, kamu akan mendapat URL seperti:
`https://namaappkamu.streamlit.app`
 - Simpan URL ini di file `url.txt` di folder submission
-

7. Kode Dashboard Lengkap

Simpan kode berikut sebagai `dashboard/dashboard.py`:

```
python
```

```

import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import os

# — Konfigurasi Halaman —————
st.set_page_config(
    page_title="Dashboard E-Commerce Analysis",
    page_icon="🛒",
    layout="wide"
)

# — Load Data —————
@st.cache_data # Cache agar data tidak di-load ulang setiap interaksi
def load_data():
    # Cari file di folder yang sama dengan dashboard.py
    base_dir = os.path.dirname(os.path.abspath(__file__))
    df = pd.read_csv(os.path.join(base_dir, 'main_data.csv'))
    df['order_purchase_timestamp'] = pd.to_datetime(df['order_purchase_timestamp'])
    df['order_delivered_customer_date'] = pd.to_datetime(df['order_delivered_customer_date'])
    df['shipping_time'] = (
        df['order_delivered_customer_date'] - df['order_purchase_timestamp']
    ).dt.days
    return df

df = load_data()

# — Sidebar Filter —————
st.sidebar.image("https://img.icons8.com/color/96/shopping-cart.png", width=80)
st.sidebar.title("🔧 Filter Data")

# Filter rentang tanggal
min_date = df['order_purchase_timestamp'].min().date()
max_date = df['order_purchase_timestamp'].max().date()

date_range = st.sidebar.date_input(
    "Rentang Tanggal",
    value=[min_date, max_date],
    min_value=min_date,
    max_value=max_date
)

# Filter metode pembayaran
payment_options = ['Semua'] + sorted(df['payment_type'].dropna().unique().tolist())

```

```

selected_payment = st.sidebar.selectbox("Metode Pembayaran", payment_options)

# Terapkan filter
df_filtered = df[
    (df['order_purchase_timestamp'].dt.date >= date_range[0]) &
    (df['order_purchase_timestamp'].dt.date <= date_range[1])
].copy()

if selected_payment != 'Semua':
    df_filtered = df_filtered[df_filtered['payment_type'] == selected_payment]

# — Header Utama —————
st.title("🛒 Dashboard Analisis E-Commerce Public Dataset")
st.markdown(
    f"Data ditampilkan: **{date_range[0]}** s.d. **{date_range[1]}** "  

    f"| Metode: **{selected_payment}**"
)
st.divider()

# — Metric Cards —————
col1, col2, col3, col4 = st.columns(4)

with col1:
    st.metric("📦 Total Order", f"{df_filtered['order_id'].nunique():,}")
with col2:
    avg_pay = df_filtered['payment_value'].mean()
    st.metric("💳 Rata-rata Pembayaran", f"R$ {avg_pay:,.2f}" if not pd.isna(avg_pay) else 0)
with col3:
    avg_ship = df_filtered['shipping_time'].mean()
    st.metric("🚚 Rata-rata Pengiriman", f"{avg_ship:.1f} hari" if not pd.isna(avg_ship) else 0)
with col4:
    total_rev = df_filtered['payment_value'].sum()
    st.metric("💰 Total Revenue", f"R$ {total_rev:,.0f}" if not pd.isna(total_rev) else 0)

st.divider()

# — Pertanyaan 1: Metode Pembayaran —————
st.header("🇮🇩 Pertanyaan 1: Analisis Metode Pembayaran")
st.markdown(
    "_Dalam periode yang dipilih, bagaimana perbedaan rata-rata nilai pembayaran "  

    "berdasarkan metode pembayaran, dan metode mana yang menghasilkan transaksi tert
)

payment_summary = (
    df_filtered
    .groupby('payment_type')
    .agg(

```

```

        avg_payment=('payment_value', 'mean'),
        total_transactions=('order_id', 'count')
    )
    .sort_values('avg_payment', ascending=False)
    .reset_index()
)

col_a, col_b = st.columns(2)

with col_a:
    fig1, ax1 = plt.subplots(figsize=(7, 4))
    colors = ['#1565C0' if i == 0 else '#90CAF9' for i in range(len(payment_summary))]
    bars = ax1.bar(payment_summary['payment_type'], payment_summary['avg_payment'],
                    color=colors, edgecolor='white')
    ax1.set_title("Rata-rata Nilai Transaksi per Metode", fontsize=12, fontweight='bold')
    ax1.set_xlabel("Metode Pembayaran")
    ax1.set_ylabel("Rata-rata (BRL)")
    ax1.tick_params(axis='x', rotation=30)
    for bar, val in zip(bars, payment_summary['avg_payment']):
        ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
                  f'{val:.1f}', ha='center', fontsize=9, fontweight='bold')
    plt.tight_layout()
    st.pyplot(fig1)
    plt.close()

with col_b:
    fig2, ax2 = plt.subplots(figsize=(7, 4))
    colors2 = ['#E65100' if i == 0 else '#FFCC80' for i in range(len(payment_summary))]
    bars2 = ax2.bar(payment_summary['payment_type'], payment_summary['total_transactions'],
                    color=colors2, edgecolor='white')
    ax2.set_title("Jumlah Transaksi per Metode", fontsize=12, fontweight='bold')
    ax2.set_xlabel("Metode Pembayaran")
    ax2.set_ylabel("Jumlah Transaksi")
    ax2.tick_params(axis='x', rotation=30)
    ax2.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x):,}'))
    plt.tight_layout()
    st.pyplot(fig2)
    plt.close()

# Kesimpulan
if not payment_summary.empty:
    top_method = payment_summary.iloc[0]['payment_type']
    top_avg = payment_summary.iloc[0]['avg_payment']
    st.success(
        f"✅ Kesimpulan: Metode {top_method} memiliki rata-rata nilai transaksi tertinggi sebesar R$ {top_avg:.2f}."
    )

```



```
st.divider()
```

```
# — Pertanyaan 2: Waktu Pengiriman per State —————
```

```
st.header("📊 Pertanyaan 2: Performa Pengiriman per Seller State")
```

```
st.markdown(
```

```
    "_Dalam periode yang dipilih, apakah lokasi seller mempengaruhi rata-rata "
```

```
    "waktu pengiriman, dan wilayah mana yang tercepat dan terlambat?_"
```

```
)
```

```
if 'seller_state' in df_filtered.columns:
```

```
    shipping_summary = (
```

```
        df_filtered
```

```
        .groupby('seller_state')
```

```
        .agg(
```

```
            avg_shipping=('shipping_time', 'mean'),
```

```
            total_orders=('order_id', 'nunique')
```

```
        )
```

```
        .sort_values('avg_shipping')
```

```
        .reset_index()
```

```
)
```

```
# Filter state representatif
```

```
min_orders = st.slider("Minimal jumlah order per state:", 10, 200, 50)
```

```
shipping_rep = shipping_summary[shipping_summary['total_orders'] >= min_orders]
```

```
col_c, col_d = st.columns([2, 1])
```

```
with col_c:
```

```
    if not shipping_rep.empty:
```

```
        fig3, ax3 = plt.subplots(figsize=(9, max(4, len(shipping_rep) * 0.35)))
```

```
        state_colors = [
```

```
            '#2E7D32' if v == shipping_rep['avg_shipping'].min()
```

```
            else '#C62828' if v == shipping_rep['avg_shipping'].max()
```

```
            else '#78909C'
```

```
            for v in shipping_rep['avg_shipping']
```

```
        ]
```

```
        ax3.barh(shipping_rep['seller_state'], shipping_rep['avg_shipping'],
```

```
                  color=state_colors, edgecolor='white')
```

```
        ax3.axvline(shipping_rep['avg_shipping'].mean(), color='navy',
```

```
                     linestyle='--', linewidth=1.2,
```

```
                     label=f'Rata-rata: {shipping_rep["avg_shipping"].mean():.1f}')
```

```
        ax3.set_title("Rata-rata Waktu Pengiriman per Seller State\n"
```

```
                       "(🟢 Tercepat | 🟡 Terlambat)", fontsize=12, fontweight='bold')
```

```
        ax3.set_xlabel("Rata-rata Waktu Pengiriman (hari)")
```

```
        ax3.set_ylabel("Seller State")
```

```
        ax3.legend(fontsize=9)
```

```

plt.tight_layout()
st.pyplot(fig3)
plt.close()

with col_d:
    if not shipping_rep.empty:
        st.markdown("***🏆 5 State Tercepat**")
        st.dataframe(
            shipping_rep.head(5)[['seller_state', 'avg_shipping', 'total_orders']
                                .rename(columns={'seller_state': 'State', 'avg_shipping': 'Rata-rata'})
                                .set_index('State'),
            use_container_width=True
        )
        st.markdown("***⚠️ 5 State Terlambat**")
        st.dataframe(
            shipping_rep.tail(5)[['seller_state', 'avg_shipping', 'total_orders']
                                .rename(columns={'seller_state': 'State', 'avg_shipping': 'Rata-rata'})
                                .set_index('State'),
            use_container_width=True
        )

# Kesimpulan
if not shipping_rep.empty:
    fastest = shipping_rep.iloc[0]
    slowest = shipping_rep.iloc[-1]
    st.success(
        f"✅ **Kesimpulan:** State **{fastest['seller_state']}** memiliki pengirim
        f"tercepat ({fastest['avg_shipping']:.1f} hari), sedangkan **{slowest['seller_state']}**
        f"yang terlambat ({slowest['avg_shipping']:.1f} hari)."
    )
else:
    st.warning("Kolom 'seller_state' tidak ditemukan. Pastikan main_data.csv sudah diupload.")

st.divider()

# — Footer —
st.caption("Dashboard Proyek Analisis Data | E-Commerce Public Dataset | Dicoding")

```

Tips & Troubleshooting

Error: `ModuleNotFoundError: No module named 'streamlit'` → Jalankan `pip install streamlit` lagi di terminal yang sama.

Error: `FileNotFoundError: main_data.csv not found` → Pastikan `main_data.csv` ada di dalam folder `dashboard/` dan kamu menjalankan streamlit dari folder `submission/`.

Dashboard tidak update setelah edit kode? → Streamlit otomatis update. Jika tidak, tekan `R` di browser atau klik "Rerun" di pojok kanan atas.

Grafik muncul dua kali? → Pastikan kamu selalu memanggil `plt.close()` setelah `st.pyplot(fig)`.

Data filter tidak bekerja? → Pastikan kolom tanggal sudah dikonversi ke `datetime` dengan `pd.to_datetime()`.